

Gravitational N-Bodies Simulation

01418343 Parallel Computing with CUDA
by 6610402230 Sirisuk Tharntham

Agenda

- **What's my project?**
- **Prepare Simulation**
- **Sequential Code**
- **Parallel Optimize**
- **Simulation**
- **Analysis & Problem**
- **Further Improve**

What's my project?



Gravitational N-Bodies simulation is for simulate the object in space by using physics equations mainly by using

Newton's law of universal gravitation

force Gravitational constant mass

$$F = G \frac{m_1 m_2}{r^2}$$

distance

The diagram shows two spheres representing masses m_1 and m_2 . m_1 is a Jupiter-like planet on the left, and m_2 is a grey, cratered sphere on the right. A horizontal line connects their centers, with the label r placed above it. Handwritten arrows point from the labels 'force', 'Gravitational constant', 'mass', and 'distance' to their respective parts in the equation above.

Prepare Simulation

Basic of Physics equations for
representing force, accerleration
and velocity

$$F = ma$$

$$s = v/t$$

$$v = u + at$$

Prepare Simulation

3D Space vector

Distance Vector
Norm
Unit-Vector

$$\vec{r}_{ij} = \vec{p}_j - \vec{p}_i$$

$$r = |\vec{r}_{ij}| = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2 + (z_j - z_i)^2}$$

$$\hat{r}_{ij} = \frac{\vec{r}_{ij}}{|\vec{r}_{ij}|}$$

Prepare Simulation

Object Force calculation

$$\vec{F}_{ij} = G \cdot \frac{m_i m_j}{r^2} \cdot \widehat{r}_{ij}$$

```
1  __host__ __device__ double getNorm(Object &oi, Object &oj){
2      Pos p1 = oi.getPosition();
3      Pos p2 = oj.getPosition();
4      Pos dp = p2 - p1;
5
6      return sqrt(dp.x*dp.x + dp.y*dp.y + dp.z*dp.z);
7  }
8
9  __host__ __device__ Pos getVectorDistance(Object &oi, Object &oj){
10     Pos p1 = oi.getPosition();
11     Pos p2 = oj.getPosition();
12     return p2 - p1;
13 }
14
15 __host__ __device__ Pos getUnitVector(Object &oi, Object &oj){
16     Pos vec = getVectorDistance(oi, oj);
17     double norm = getNorm(oi, oj);
18     if(norm == 0) return Pos(0, 0, 0);
19     return vec / norm;
20 }
21
22 __host__ __device__ Pos getForce(Object &oi, Object &oj){
23     double r = getNorm(oi, oj);
24     double F = G * (oi.m * oj.m) / (r * r + 1e-6f);
25     Pos direction = getUnitVector(oi, oj);
26     return direction * F;
27 }
```


Sequential Code

Pseudocode (to calculate force i,j)

```
1  for object i from 0 to n-1
2      total_force ← array of n force affect from all i,j for each object
3      for object j from 0 to n-1
4          if i != j
5              total_force[i] ← total_force[i] + getForce(obj_i,obj_j)
6      object[i] ← update a, v and position
7      object[i].a ← F/m
8      object[i].v ← u + a * dt
9      object[i].pos ← object[i].pos + v * dt
```


Sequential Code

Pseudocode (to calculate force i,j)

```
1 for object i from 0 to n-1
2   total_force ← array of n force affect from all i,j for each object
3   for object j from 0 to n-1
4     if i != j
5       total_force[i] ← total_force[i] + getForce(obj_i,obj_j)
6   object[i] ← update a, v and position
7   object[i].a ← F/m
8   object[i].v ← u + a * dt
9   object[i].pos ← object[i].pos + v * dt
```



$O(n^2)$

Sequential Code

```
1 void computeStepCPU(){
2     vector<Pos> totalForce(objList.size(), Pos(0,0,0));
3     //UPDATE FORCE
4     for(int i = 0;i < objList.size();++i){
5         for(int j = 0;j < objList.size();++j){
6             if(i == j)continue;
7             totalForce[i] = totalForce[i] + physicsEngine.getForce(objList[i],objList[j]);
8         }
9     }
10    //UPDATE A V POS
11    for(int i = 0;i < objList.size();++i){
12        Object &obj = objList[i];
13        obj.a = totalForce[i] / obj.m;
14        obj.v = obj.v + obj.a * dt;
15        obj.pos = obj.pos + obj.v * dt;
16    }
17
18 }
```


Parallel Optimize

Idea

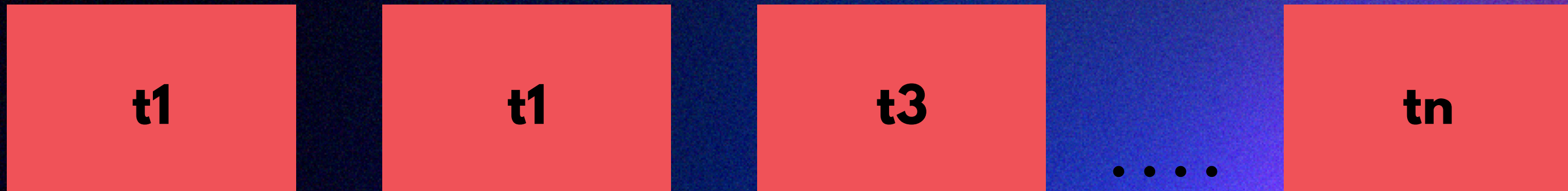
1. force computation is separate
2. updated value is a, v , and pos
3. updated process is distinct

Limitation

no way to do nested kernel

Parallel Optimize

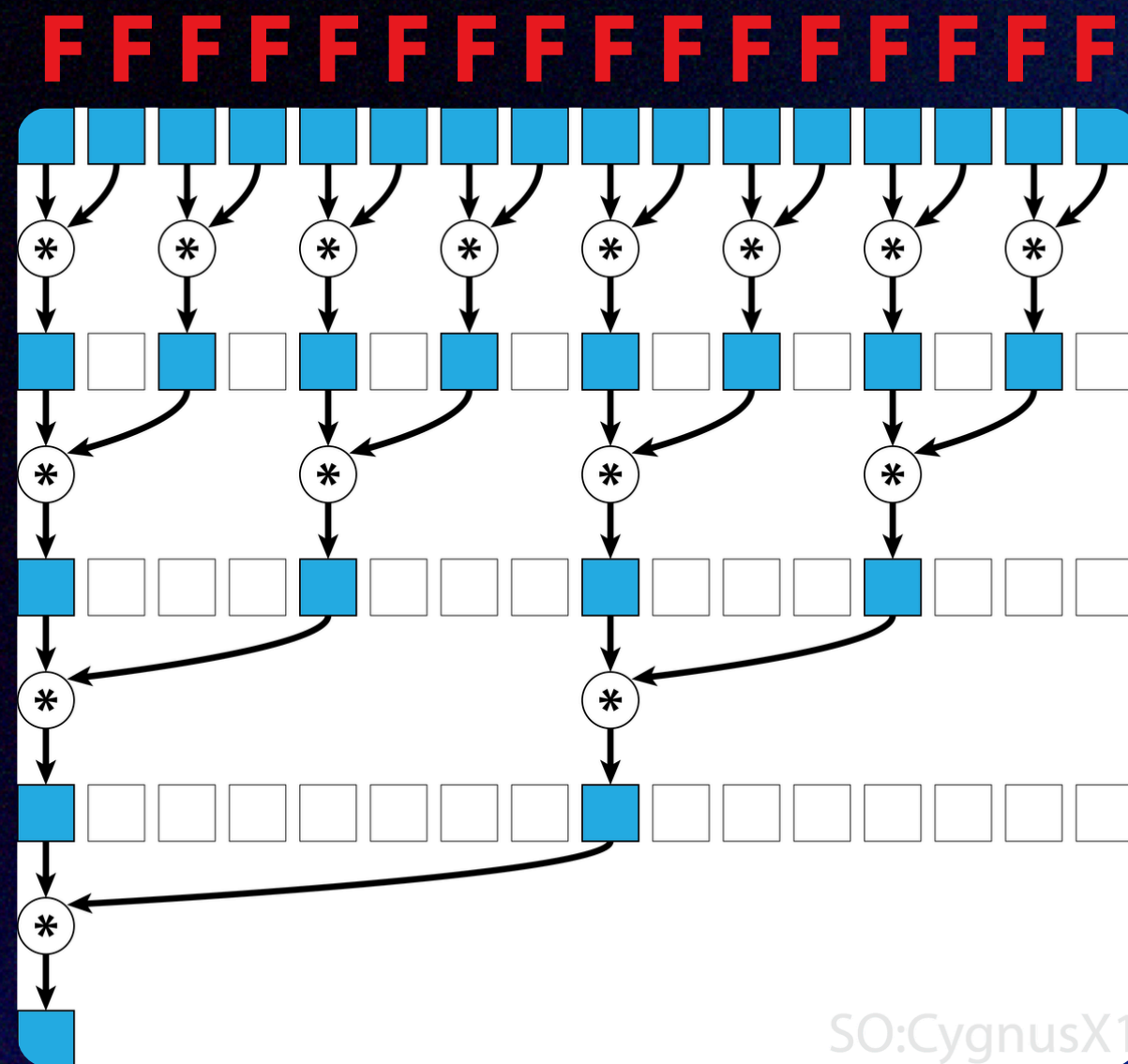
**My design
to plug-in some parallel subroutine**



**the processor can do calcualte force by
it self? → YES**

Parallel Optimize

With the addition operation have association properties
then reduction is work



I've designed each thread carry
do the force calculation then SUM

Parallel Optimize

UPDATE Accerlertation, Velocity and Position



update operation can be done by threads simutaneously

Parallel Optimize

Pseudocode (to calculate force i,j)

```
1 for object i from 0 to n-1
2     total_force ← array of n force affect from all i,j for each object
3     forall object j from 0 to n-1
4         if i != j
5             total_force[i] ← total_force[i] + getForce(obj_i,obj_j)
6 forall object i from 0 to n-1
7     object[i] ← update a, v and position
```


Parallel Optimize

```
1  __global__ void computeObjectKernel(Object *d_objList, Pos *d_total_force, int size, double dt){  
2      int idx = blockDim.x * blockIdx.x + threadIdx.x;  
3      if(idx < size){  
4          Object &obj = d_objList[idx];  
5          obj.a = d_total_force[idx] / obj.m;  
6          obj.v = obj.v + obj.a * dt;  
7          obj.pos = obj.pos + obj.v * dt;  
8      }  
9  }
```

map p processors to calculate


```

1  __global__ void reduceForceKernel(int curForceIdx, Object *d_objList, Pos *d_total_force, Physics *physicsEngine, int size){
2      __shared__ Pos ds_force[BLOCK_SIZE];
3      int t = threadIdx.x;
4      int total_threads = gridDim.x * blockDim.x;
5      int part = (size + total_threads - 1) / total_threads;
6
7      Pos sum(0, 0, 0);
8
9      for(int p = 0; p < part; ++p){
10         int start = p * total_threads;
11         int idx = start + blockDim.x * blockIdx.x + threadIdx.x;
12
13         if(idx < size && idx != curForceIdx){
14             ds_force[t] = physicsEngine->getForce(d_objList[curForceIdx], d_objList[idx]);
15         }else{
16             ds_force[t] = Pos(0, 0, 0);
17         }
18         __syncthreads();
19
20         for(int s = BLOCK_SIZE / 2; s > 0; s >>= 1){
21             if(t < s){
22                 ds_force[t] = ds_force[t] + ds_force[t + s];
23             }
24             __syncthreads();
25         }
26
27         if(t == 0) sum = sum + ds_force[0];
28         __syncthreads();
29     }
30
31     if(t == 0) d_total_force[curForceIdx] = sum;
32 }

```

reduction kernel

Parallel Optimize

```
Loaded config from config.txt
Loaded 10001 objects from object_input.csv
=== BENCHMARK MODE ===
Objects: 10001
Steps: 600
Time per step: 1 days

Running CPU simulation...
  CPU Progress: 590/600
CPU Time: 2083.02 seconds
CPU Speed: 0.288043 steps/second

Running GPU simulation...
  GPU Progress: 590/600
GPU Time: 42.2612 seconds
GPU Speed: 14.1974 steps/second

=== RESULTS ===
CPU Time: 2083.02 s
GPU Time: 42.2612 s
Speedup: 49.2891x
```

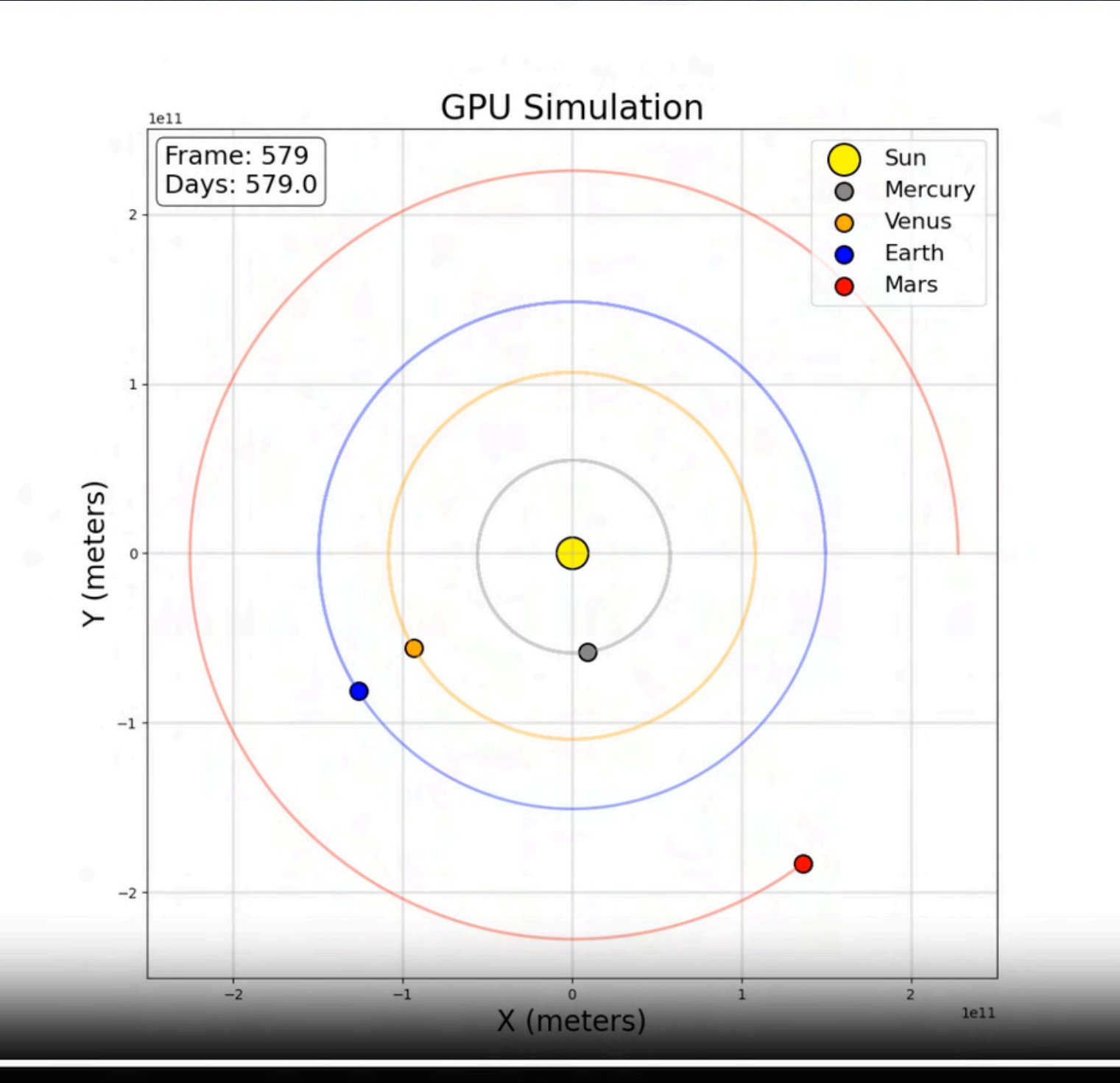
**> 2X Faster
in practice**

Simulation

2 MAIN Components

Dynamic Objects Calculation (CUDA)

```
output_gpu.csv
1 FRAME,NAME,MASS,VELOCITY,ACCELERATION,POSITION
2 0,Sun,1.989e+30,(0,0,0),(0,0,0),(0,0,0)
3 0,Mercury,3.285e+23,(0,47400,0),(0,0,0),(5.79e+10,0,0)
4 0,Venus,4.867e+24,(0,35020,0),(0,0,0),(1.082e+11,0,0)
5 0,Earth,5.972e+24,(0,29783,0),(0,0,0),(1.496e+11,0,0)
6 0,Mars,6.39e+23,(0,24070,0),(0,0,0),(2.279e+11,0,0)
7 1,Sun,1.989e+30,(0.00457211,0,0),(5.29179e-08,0,0),(395.03,0,0)
8 1,Mercury,3.285e+23,(-3421.33,47400,0),(-0.0395987,0,0),(5.76044e+10,4.09536e+09,0)
9 1,Venus,4.867e+24,(-979.695,35020,0),(-0.0113391,0,0),(1.08115e+11,3.02573e+09,0)
10 1,Earth,5.972e+24,(-512.513,29783,0),(-0.00593186,0,0),(1.49556e+11,2.57325e+09,0)
11 1,Mars,6.39e+23,(-220.841,24070,0),(-0.00255604,0,0),(2.27881e+11,2.07965e+09,0)
12 2,Sun,1.989e+30,(0.0091469,0.000134526,0),(5.2949e-08,1.55701e-09,0),(1185.32,11.6231,0)
13 2,Mercury,3.285e+23,(-6851.83,47156.1,0),(-0.0397048,-0.00282281,0),(5.70124e+10,8.16965e+09,0)
14 2,Venus,4.867e+24,(-1959.77,34992.6,0),(-0.0113435,-0.000317469,0),(1.07946e+11,6.04909e+09,0)
15 2,Earth,5.972e+24,(-1025.1,29774.2,0),(-0.00593274,-0.000102073,0),(1.49467e+11,5.14574e+09,0)
16 2,Mars,6.39e+23,(-441.692,24068,0),(-0.00255614,-2.3326e-05,0),(2.27843e+11,4.15912e+09,0)
17 3,Sun,1.989e+30,(0.0137192,0.000404142,0),(5.29205e-08,3.12056e-09,0),(2370.66,46.541,0)
18 3,Mercury,3.285e+23,(-10274.6,46665.6,0),(-0.0396149,-0.00567669,0),(5.61247e+10,1.22016e+10,0)
19 3,Venus,4.867e+24,(-2939.47,34937.7,0),(-0.011339,-0.000635435,0),(1.07692e+11,9.0677e+09,0)
20 3,Earth,5.972e+24,(-1537.61,29756.5,0),(-0.00593186,-0.000204207,0),(1.49334e+11,7.71671e+09,0)
```



Video Generator (Python)

Analysis & Problem

Theoretical analysis with work-span model

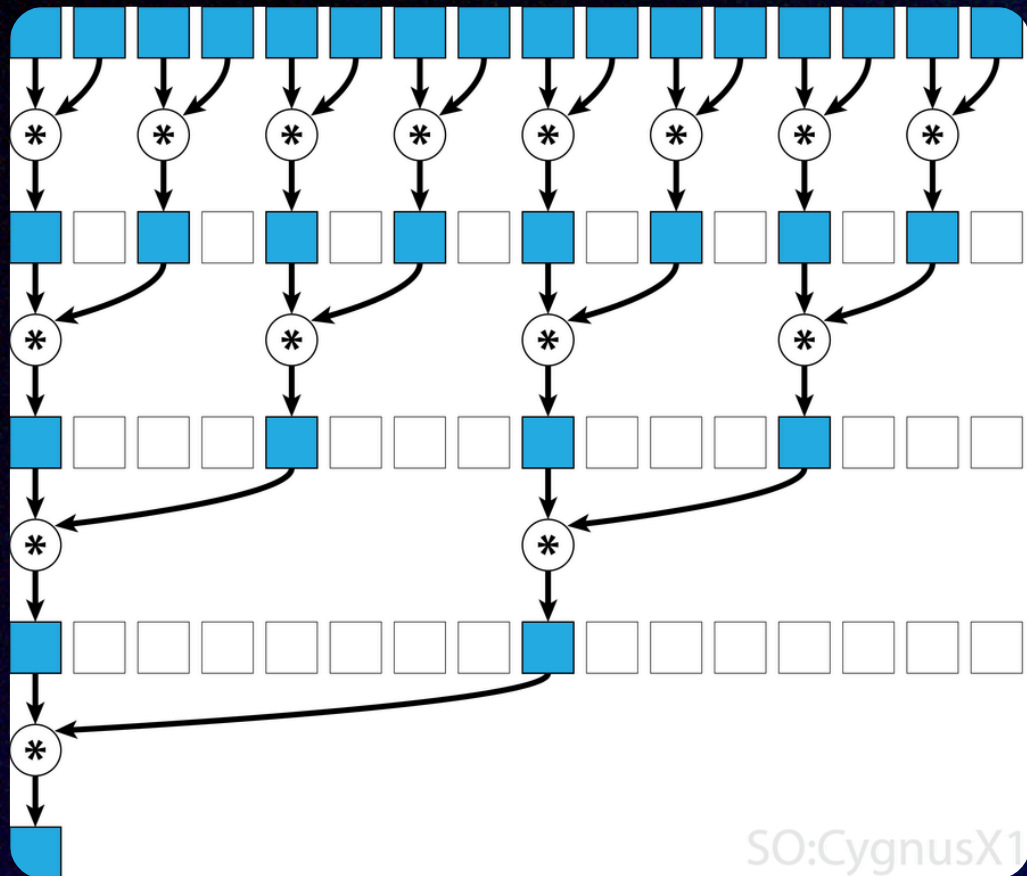
```
1  for object i from 0 to n-1
2      total_force ← array of n force affect from all i,j for each object
3      forall object j from 0 to n-1
4          if i != j
5              total_force[i] ← total_force[i] + getForce(obj_i,obj_j)
6  forall object i from 0 to n-1
7      object[i] ← update a, v and position
```

REDUCTION

MAP

Analysis & Problem

The reduction have $T(n) = w(n)/p + \log n$



best possible when p is equal to n
span perform in $O(\log n)$
 $T(n) = O(\log n)$

$O(n)$

for object i from 0 to $n-1$
REDUCTION_KERNEL

outer loop could perform
kernel n times
then total time complexity is
 $O(n \log n)$

Speedup (T_s/T_p) $\rightarrow$
 $O(n)/O(\log n)$ times

Analysis & Problem

more MemCpy with small size

```
Loaded config from config.txt
Loaded 5 objects from object_input.csv
=== BENCHMARK MODE ===
Objects: 5
Steps: 600
Time per step: 1 days

Running CPU simulation...
  CPU Progress: 590/600
CPU Time: 0.001 seconds
CPU Speed: 600000 steps/second

Running GPU simulation...
  GPU Progress: 590/600
GPU Time: 0.259658 seconds
GPU Speed: 2310.73 steps/second

=== RESULTS ===
CPU Time: 0.001 s
GPU Time: 0.259658 s
Speedup: 0.00385122x
```

less MemCpy with small size

```
Loaded config from config.txt
Loaded 5 objects from object_input.csv
=== BENCHMARK MODE ===
Objects: 5
Steps: 600
Time per step: 1 days

Running CPU simulation...
  CPU Progress: 590/600
CPU Time: 0.001 seconds
CPU Speed: 600000 steps/second

Running GPU simulation...
  GPU Progress: 590/600
GPU Time: 0.0214723 seconds
GPU Speed: 27943 steps/second

=== RESULTS ===
CPU Time: 0.001 s
GPU Time: 0.0214723 s
Speedup: 0.0465717x
```


Analysis & Problem

> 0.5 slower when always initialize & mem transfer

```
Loaded config from config.txt
Loaded 1001 objects from object_input.csv
=== BENCHMARK MODE ===
Objects: 1001
Steps: 100
Time per step: 1 days
```

```
Running CPU simulation...
  CPU Progress: 90/100
CPU Time: 3.496 seconds
CPU Speed: 28.6041 steps/second
```

```
Running GPU simulation...
  GPU Progress: 90/100
GPU Time: 8.3974 seconds
GPU Speed: 11.9084 steps/second
```

```
=== RESULTS ===
CPU Time: 3.496 s
GPU Time: 8.3974 s
Speedup: 0.416319x
```

> 2X Faster after reduce MemCpy

```
Loaded config from config.txt
Loaded 1001 objects from object_input.csv
=== BENCHMARK MODE ===
Objects: 1001
Steps: 100
Time per step: 1 days
```

```
Running CPU simulation...
  CPU Progress: 90/100
CPU Time: 3.471 seconds
CPU Speed: 28.8101 steps/second
```

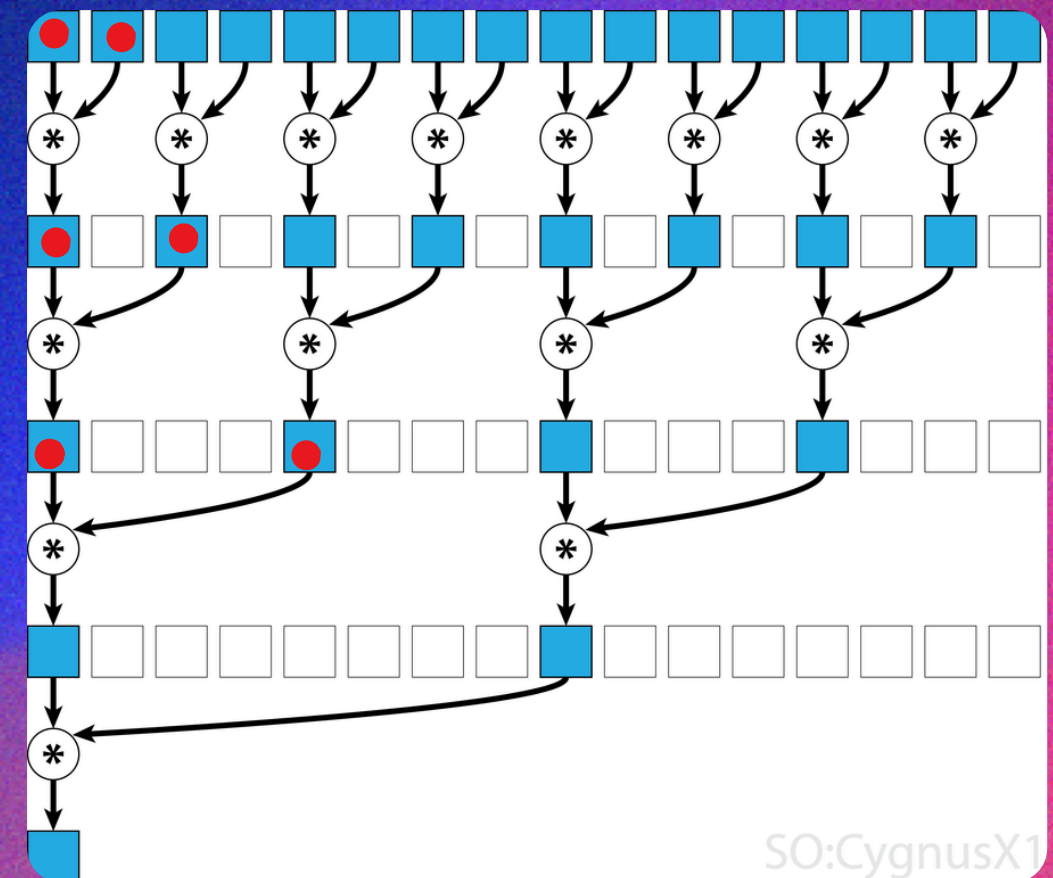
```
Running GPU simulation...
  GPU Progress: 90/100
GPU Time: 0.547663 seconds
GPU Speed: 182.594 steps/second
```

```
=== RESULTS ===
CPU Time: 3.471 s
GPU Time: 0.547663 s
Speedup: 6.33784x
```


Analysis & Problem

- floating point comparison problems due to IEEE 754 mantissa have limit precision and when handling operation with very small value cause estimate after many times of operation values not always equal

- The fact is sequential algorithm performs operation in linear
- But reduction perform operations with stride cause values not easily to compare with epsilon floating point comparing method



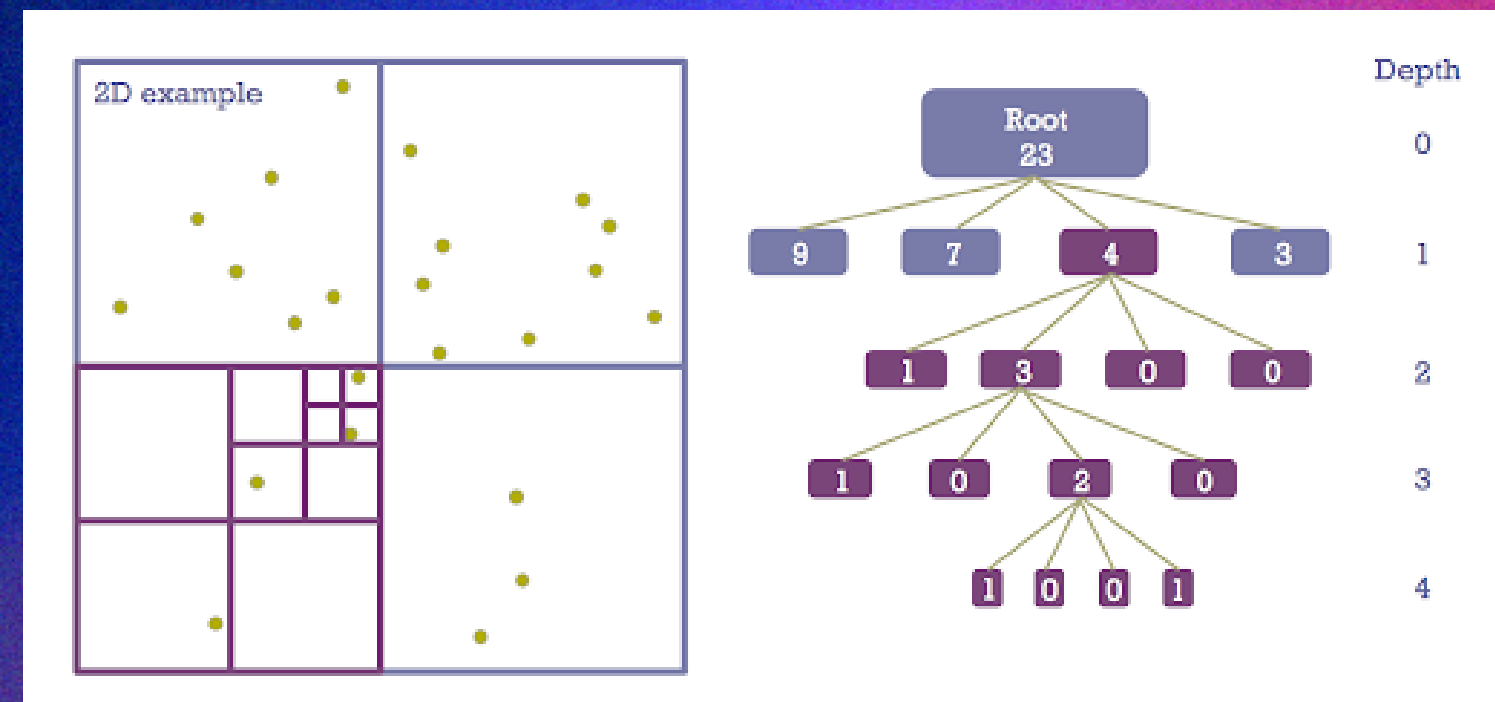
Further Improve

- Reactor code structure
- handling odd block_size to 2^k
- handling better floating point comparison
- prevent memory transfer processes
- new algorithm to reduce work (ws-model)

My Research found some new way.

Barnes-Hut (Tree)

try to estimate far object as one



Thank You!

6610402230 Sirisuk Tharntham Section 1